

Detección de Adaptativa de Carriles en Condiciones de Iluminación Variables

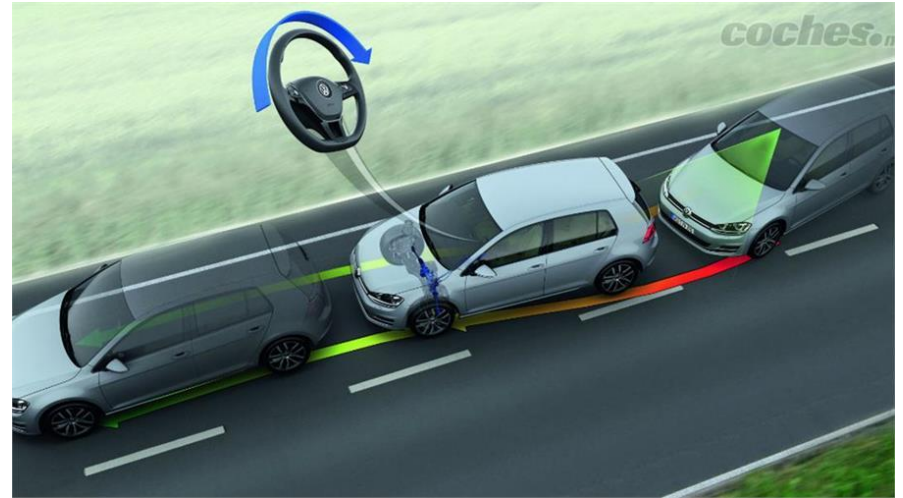
Daniel Salamanca Garrido
Hugo Toro Meca
Miguel García Losada

CONTENIDO

1. INTRODUCCIÓN
2. PLANTEAMIENTO TEÓRICO
3. IMPLEMENTACIÓN
4. EXPERIMENTACIÓN
5. CONCLUSIONES

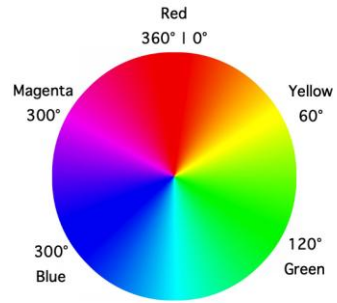
INTRODUCCIÓN

LOS COCHES AUTÓNOMOS Y ASISTENCIA A LA CONDUCCIÓN



PLANTEAMIENTO TEÓRICO

HLS



Hue
(red, green, etc.)



Saturation
(essence of the hue)



Lightness
(darkness/brightness)

ESCALA DE GRISES



FILTRADO GAUSSIANO



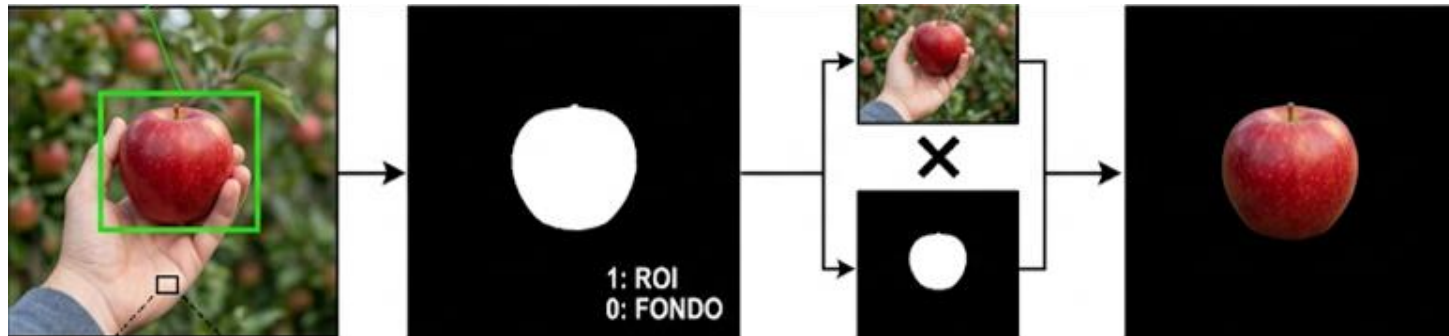
$1 / 16$

1	2	1
2	4	2
1	2	1

Por cada pixel si aplicamos una máscara 3x3



ROI



Filtro de Canny – Paso a Paso

1 Imagen original

Partimos de la imagen en escala de grises.



2 Suavizado Gaussiano

Se aplica un filtro Gaussiano para reducir el ruido.



Ejemplo de kernel Gaussiano 5x5

1	4	6	4	1
4	16	24	16	4
6	24	36	24	6
4	16	24	16	4
1	4	6	4	1

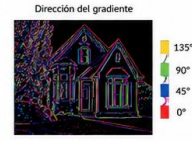
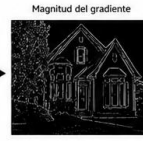
(se normaliza dividiendo por la suma)

¿Qué hace?

Elimina ruido y pequeñas variaciones de intensidad para evitar detectar bordes falsos.

3 Cálculo del gradiente

Se calcula la magnitud y dirección del gradiente (usando operadores Sobel, por ejemplo).

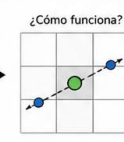


¿Qué hace?

Resalta las zonas con cambios de intensidad y obtiene la dirección de esos cambios.

4 Supresión de no-máximos

Se conservan solo los píxeles que son máximos locales en la dirección del gradiente. Esto deja bordes más delgados.



Se compara el píxel central con sus vecinos en la dirección del gradiente. Si no es el máximo, se suprime (se pone a 0).

¿Qué hace?

Elimina bordes gruesos y conserva solo la línea del borde.

5 Umbralización con histeresis

Se usan dos umbrales: alto y bajo.

- Si la magnitud > umbral alto: borde fuerte (se conserva).
- Si está entre umbral bajo y alto: se conserva solo si está conectado a un borde fuerte.
- Si es < umbral bajo: se elimina.

Después de supresión (no umbralizado)



Después de histeresis



Ejemplo de umbrales

- > Umbral alto (borde fuerte)
- Entre umbral bajo y alto (se conserva si está conectado)
- < Umbral bajo (se elimina)

Alta
Baja

¿Qué hace?

Obtiene bordes continuos y elimina respuestas débiles aisladas.

6 Resultado final

Mapa de bordes detectados.



Características del filtro de Canny

- ✓ Detecta bordes reales con buena localización.
- ✓ Produce bordes delgados.
- ✓ Es robusto al ruido.

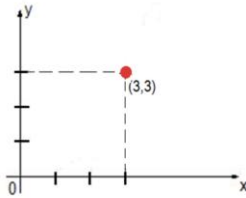
Parámetros importantes

- Tamaño y desviación del Gaussiano (suavizado).
- Umbral alto y umbral bajo.

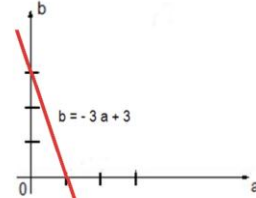
Transformada de Hough Estándar

$$y_i = a x_i + b$$

$$b = -x_i a + y_i$$



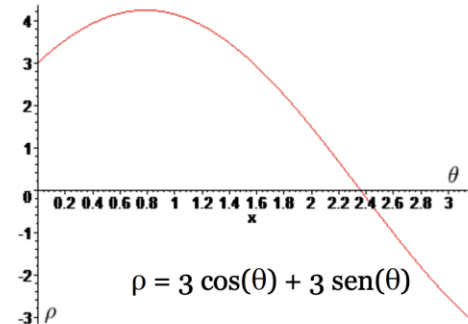
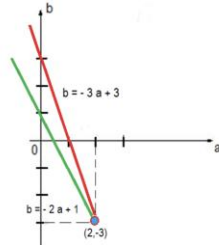
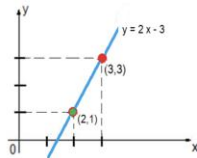
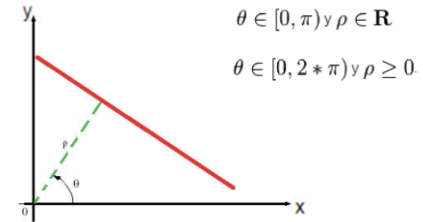
Plano xy

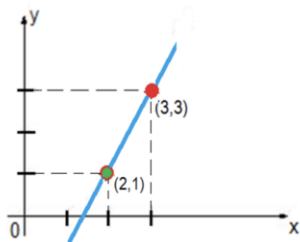


Espacio paramétrico ab

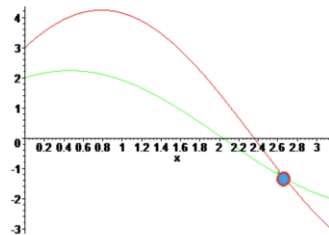


$$\rho = x \cos(\theta) + y \sin(\theta)$$

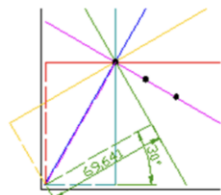
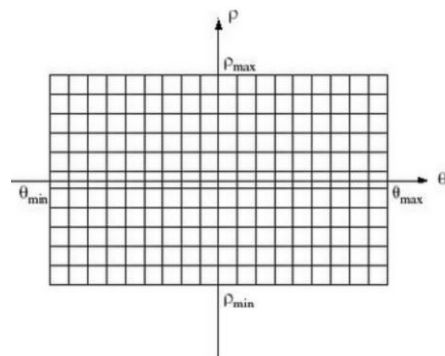




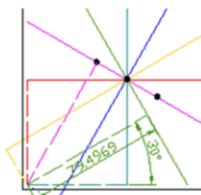
Plano xy



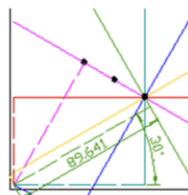
Espacio paramétrico $\theta\rho$



Angle	Dist.
0	40
30	69.6
60	81.2
90	70
120	40.6
150	0.4



Angle	Dist.
0	57.1
30	79.5
60	80.5
90	60
120	23.4
150	-19.5



Angle	Dist.
0	74.6
30	89.6
60	80.6
90	50
120	6.0
150	-39.6

	0	30	60	90	120	150
100	0	0	0	0	0	0
90	0	1	0	0	0	0
80	0	1	3	0	0	0
70	1	1	0	1	0	0
60	1	0	0	1	0	0
50	0	0	0	1	0	0
40	1	0	0	0	1	0
30	0	0	0	0	0	0
20	0	0	0	0	1	0
10	0	0	0	0	1	0
0	0	0	0	0	0	1
-10	0	0	0	0	0	0
-20	0	0	0	0	0	1
-30	0	0	0	0	0	0
-40	0	0	0	0	0	1

La recta de parámetros

$$\rho=80$$

$$\theta=60$$

contiene a los tres puntos de borde analizados.

FILTRO GAMMA ADAPTATIVO

1. Imagen original



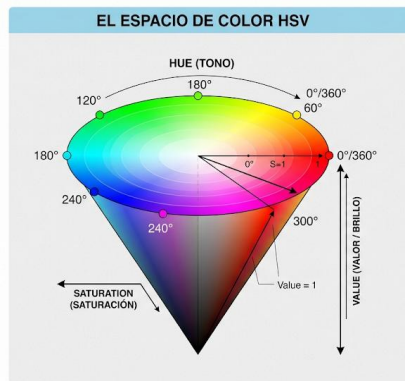
3. Imagen corregida con gamma



$$\gamma_{base} = \frac{-0,3}{\log_{10}(Y_{average} + \epsilon)}$$

$$\gamma = 0,7 - \gamma_{base}$$

HSV



1. AISLAMIENTO DE HUE (TONO)



2. AISLAMIENTO DE SATURATION (SATURACIÓN)



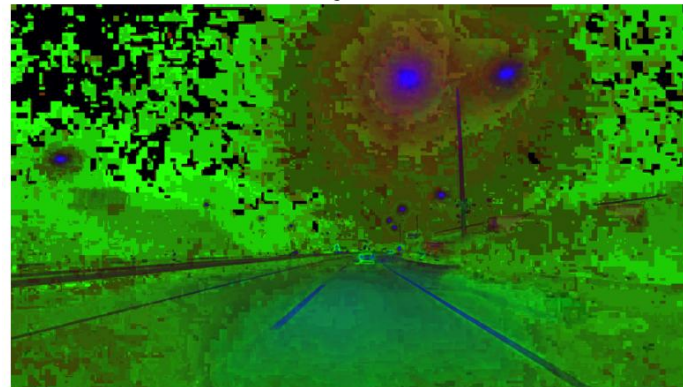
3. AISLAMIENTO DE VALUE (VALOR)



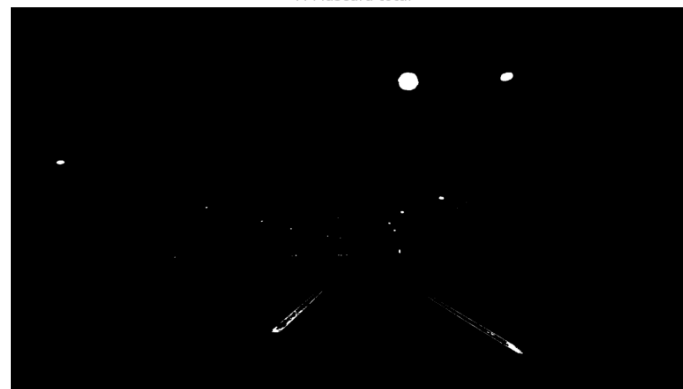
HSV PARA SEGMENTACIÓN DE COLOR



4. Imagen en HSV



7. Máscara total



$$mask = mask_{yellow} \vee mask_{white}$$

$$I_{filtered} = I_{RGB} \cdot mask$$

Transformada de Hough Probabilística

$$H(\rho, \theta) = \sum I_{edges}(x, y)$$

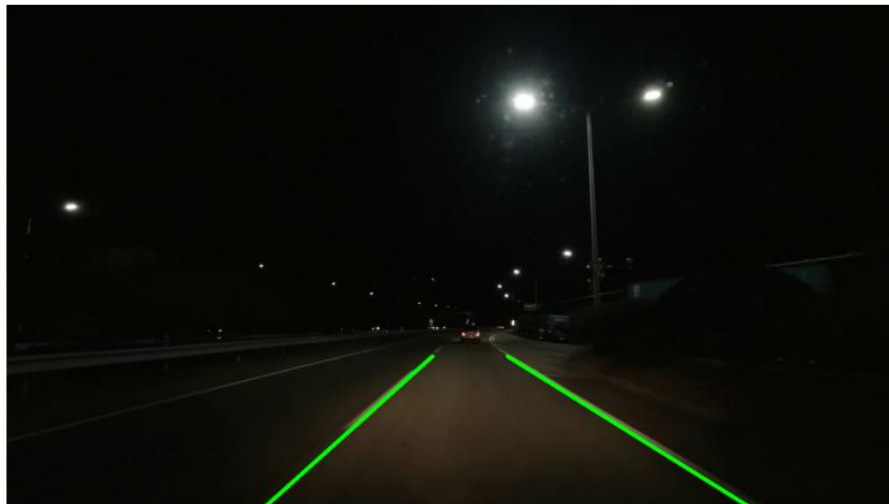
$$\theta = \left| \arctan \left(\frac{y_2 - y_1}{x_2 - x_1} \right) \right| \quad m = \frac{y_2 - y_1}{x_2 - x_1}, \quad c = y_1 - mx_1$$
$$x_m = \frac{x_1 + x_2}{2}$$
$$d = |x_m - x_c|$$

Resultados PHT

10. Segmentos detectados por Hough probabilística

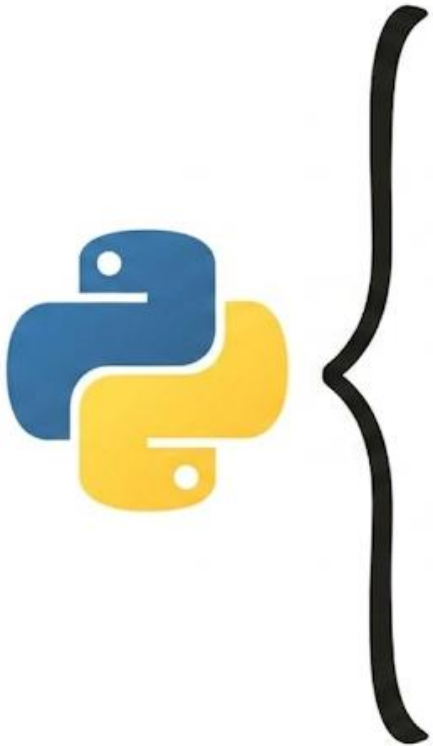


11. Resultado final



IMPLEMENTACIÓN

Librerías



Clasificación Dinámica de Iluminación

¿B?

>50

<50



Clasificador

Cambio a escala de grises:

```
imagen_gris = cv2.cvtColor(imagen, cv2.COLOR_BGR2GRAY)
```

Cálculo de media:

```
brillo_medio = np.mean(imagen_gris)
```

$$B = \frac{1}{M \times N} \sum_{i=1}^M \sum_{j=1}^N I_{gray}(i, j)$$

M y N = alto y ancho

I (i,j) = intensidad (0-255)

$$F(B) = \begin{cases} 1 & \text{si } B > 50 \implies \text{MODO DÍA} \\ 0 & \text{si } B \leq 50 \implies \text{MODO NOCHE} \end{cases}$$

Pipeline Diurno

Filtro de Color, Escala de Grises y Suavizado

1. Original



2. Selección de Color



3. Grises + Gaussian Blur



Flitro de Color

Conversión HSV:

```
hsv = cv2.cvtColor(img_dia, cv2.COLOR_BGR2HSV)
```

Filtramos la imagen a partir de 2 máscaras:

- Máscara Amarilla:

```
cv2.inRange(hsv, np.array([15, 80, 80]),  
np.array([35, 255, 255]))
```

- Máscara Blanca:

```
cv2.inRange(hsv, np.array([0, 0, 200]),  
np.array([180, 30, 255]))
```

Aplicación de ambas máscaras:

```
cv2.bitwise_and(img_dia,  
img_dia,mask=mascara_color)
```

1. Original



2. Selección de Color



Filtro Gaussiano

Conversión a escala de Grises:

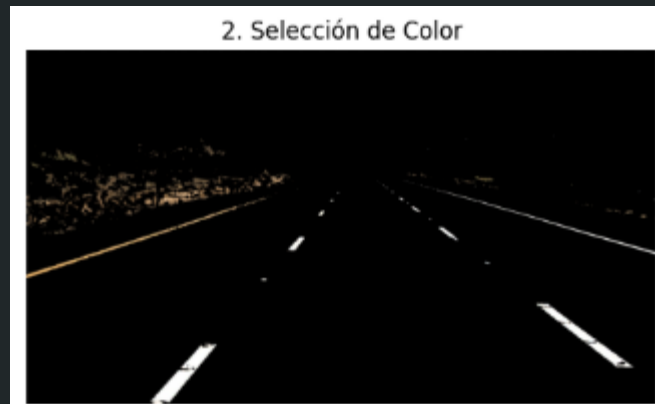
```
cv2.cvtColor(img_filtrada_color, cv2.COLOR_BGR2GRAY)
```

Suavizado Gaussiano:

```
cv2.GaussianBlur(img_gris, (5, 5), 0)
```

(5,5) => tamaño del Kernel

0 => sigmaX, expansión de función gaussiana



Detección de Bordos y Regiones de Interés(ROI)

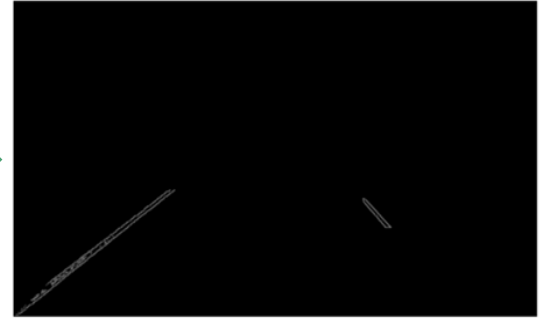
4. Canny Edge Detector



5. Mascara ROI (Ensanchada)



6. Bordos Recortados



Detección de Bordos

Aplicación de Canny:

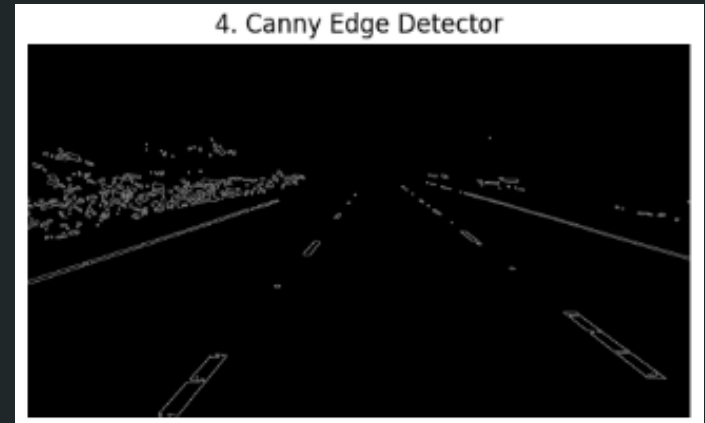
```
img_canny = cv2.Canny(img_blur, 50, 150)
```

gradiente $> 150 \rightarrow$ borde seguro.

gradiente $< 50 \rightarrow$ se elimina.

$50 \leq \text{gradiente} \leq 150 \rightarrow$ depende.

Solo se conserva si está conectado a un borde fuerte.



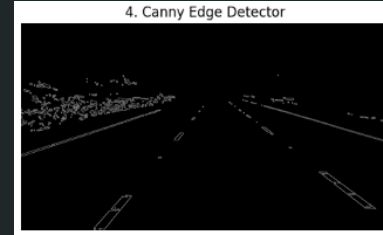
Región de interés

Creación del polígono:

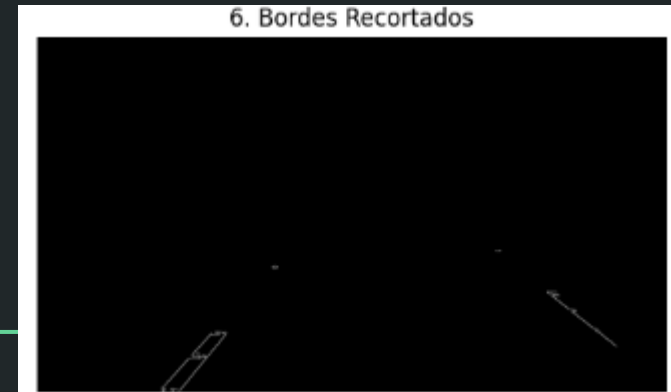
```
np.array([[
    (0, alto), (ancho, alto),
    (int(ancho * 0.75), int(alto * 0.6)),
    (int(ancho * 0.25), int(alto * 0.6))
]])
```

Filtramos la imagen con la máscara ROI:

```
img_roi = cv2.bitwise_and(img_canny, mascara_roi)
```

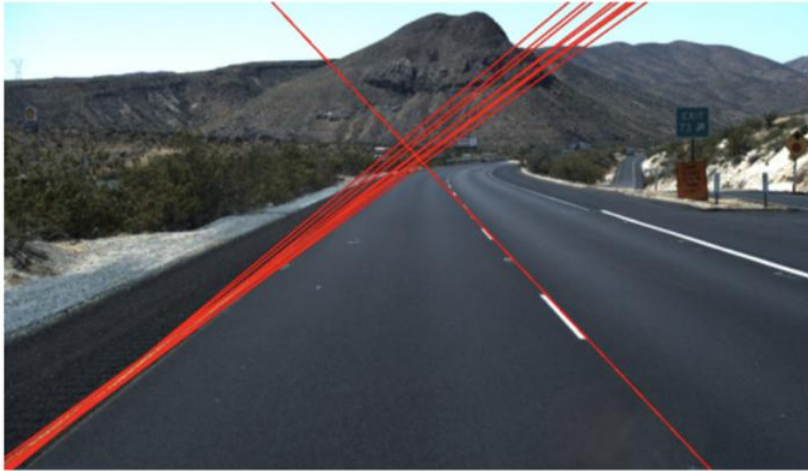


X

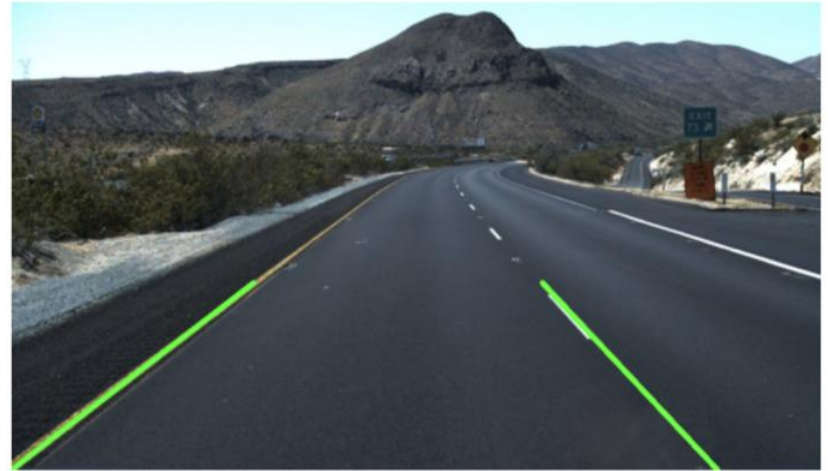


Hough Estándar y Lane Line Averaging

7. Hough Estándar (Líneas Infinitas + Trigonometría)



8. Final: Lane Line Averaging (Eq. 9 y 10)



Hough (SHT)

Aplicación de Hough:

```
cv2.HoughLines(img_roi, rho=1, theta=np.pi/180,  
threshold=80)
```

threshold => número de votos necesarios

Transformación Polar -> Cartesiano:

```
slope = -math.cos(theta) / math.sin(theta)  
intercept = rho / math.sin(theta)
```

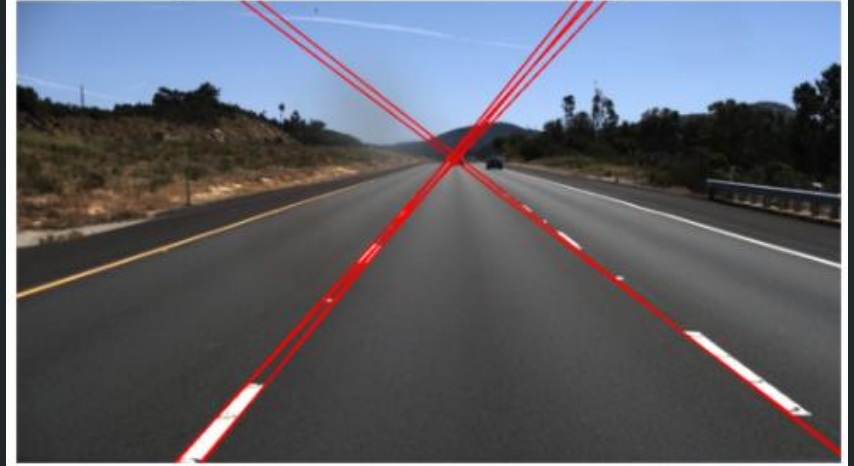
Suposición de puntos artificiales:

```
x0 = math.cos(theta) * rho  
y0 = math.sin(theta) * rho  
x1_draw = int(x0 + 1500 * (-math.sin(theta)))  
y1_draw = int(y0 + 1500 * (math.cos(theta)))  
x2_draw = int(x0 - 1500 * (-math.sin(theta)))  
y2_draw = int(y0 - 1500 * (math.cos(theta)))  
cv2.line(img_hough_bruto, (x1_draw, y1_draw),  
(x2_draw, y2_draw), (255, 0, 0), 2)
```

Filtrado de líneas:

```
if slope < -0.5:  
    pendientes_izq.append(slope)  
    interceptos_izq.append(intercept)  
elif slope > 0.5:  
    pendientes_der.append(slope)  
    interceptos_der.append(intercept)
```

7. Hough Estándar (Líneas Infinitas + Trigonometría)



Promediado de Líneas

Calculamos mediante la media:

```
avg_slope = np.mean(pendientes)  
avg_intercept = np.mean(interceptos)
```

Coordenadas x, y:

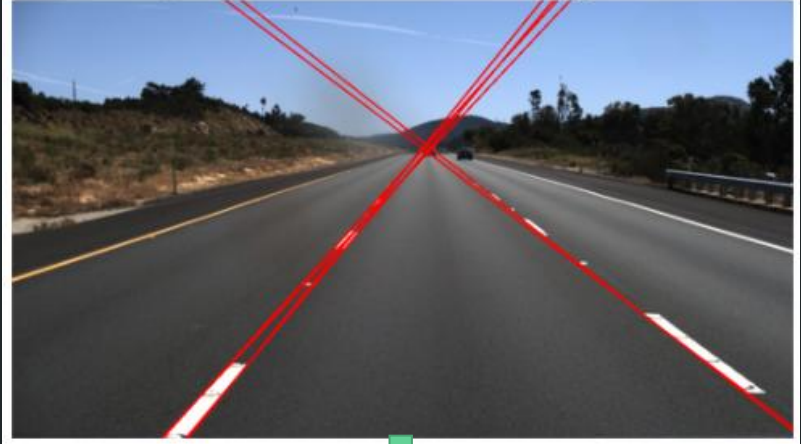
```
y_max = alto  
y_min = int(alto * 0.6)
```

```
x_fondo = int((y_min - avg_intercept) / avg_slope)  
x_base = int((y_max - avg_intercept) / avg_slope)
```

Resultado de la línea:

```
cv2.line(img_resultado_final, (x_fondo, y_min),  
(x_base, y_max), color, 8)
```

7. Hough Estándar (Líneas Infinitas + Trigonometría)



8. Final: Lane Line Averaging (Eq. 9 y 10)



Pipeline Nocturno

Corrección Gamma Adaptativa, Filtrado HSV y ROI

Original (Y_{avg} : 20.8)



Gamma Adaptativa (γ : 0.93)



Filtro HSV + ROI



Corrección de Gamma

Conversión escala de grises:

```
gris_noche = cv2.cvtColor(img_noche,  
cv2.COLOR_BGR2GRAY)
```

Fórmula de gamma adaptativo:

```
y_average = np.mean(gris_noche)  
epsilon = 1e-6  
gamma_base = -0.3 / math.log10(y_average + epsilon)  
gamma_final = 0.7 - gamma_base
```

Aplicación de la fórmula mediante LUT:

```
inv_gamma = 1.0 / gamma_final  
tabla = np.array([((i / 255.0) ** inv_gamma) * 255  
for i in np.arange(0, 256)]).astype("uint8")  
img_gamma = cv2.LUT(img_noche, tabla)
```

$$\text{Píxel}_{\text{nuevo}} = \left(\frac{\text{Píxel}_{\text{original}}}{255} \right)^{\frac{1}{\gamma}} \times 255$$

Original (Y_avg: 14.5)



Gamma Adaptativa (γ: 0.96)



Filtro de Color + Región de interés

De manera similar al pipeline diurno pero con unos valores más restrictivos

```
rango_bajo_blanco = np.array([0, 0, 180])  
rango_alto_blanco = np.array([180, 30, 255])  
rango_bajo_amarillo = np.array([15, 80, 80])  
rango_alto_amarillo = np.array([35, 255, 255])
```

```
mascara_color = cv2.bitwise_or(mask_white,  
mask_yellow)
```

Filtramos la imagen con la máscara ROI:

```
mascara_binaria_final =  
cv2.bitwise_and(mascara_color, mask_roi)
```

Gamma Adaptativa (γ : 0.96)



Filtro HSV + ROI



Transformada Probabilística de Hough y Detección de Límites



Hough (PHT)

Aplicación de Hough Probabilística:

```
cv2.HoughLinesP(mascara_binaria_final,rho=1,theta=
np.pi/180,
threshold=20,minLineLength=20,maxLineGap=150)
```

- **Threshold (Umbral de Votos):** Número mínimo de píxeles alineados necesarios para considerar que existe una recta.
- **Min Line Length(Longitud Mínima):** La criba de tamaño. Si un segmento detectado mide menos que este valor, se borra. (Perfecto para ignorar el ruido).
- **Max Line Gap(Brecha Máxima):** Distancia máxima permitida entre dos trazos para que el algoritmo decida fusionarlos en una sola línea continua.

Cálculo de Pendiente e intersección:

```
m = (y2 - y1) / (x2 - x1)
c = y1 - m * x1
```

Cálculo de mejor segmento (más cercano al centro):

```
xm = (x1 + x2) / 2
d = abs(xm - centro_x)
```

```
mejor = min(candidatos,key=lambda x:x['d'])
m, c = mejor['m'], mejor['c']
```

Líneas candidatas detectadas y escaladas (Filtro por d)



EXPERIMENTACIÓN

Métodos de Validación

Detección de líneas (Detection Rate): Representa la tasa de éxito bruto del algoritmo de percepción. Se define como el porcentaje o ratio de fotogramas en los que el sistema es capaz de localizar e instanciar **simultáneamente** tanto el límite izquierdo como el derecho del carril. Un fallo en cualquiera de las dos líneas computa el fotograma como "no detectado".

Consistencia (Consistency): Mide la estabilidad temporal e inercial del trazado calculado. Para que un fotograma se considere "consistente", la variación matemática de la pendiente de ambas líneas no debe oscilar más de un **15%** respecto al fotograma inmediatamente anterior.

Distancia (Distance): Cuantifica la separación (medida en píxeles) entre el carril izquierdo y el carril derecho en la base inferior de la imagen (la zona de la carretera más próxima al capó del vehículo). Esta métrica permite validar que el sistema mantiene una anchura de carril constante y realista desde la perspectiva de la cámara, asegurando que las líneas detectadas no convergen ni divergen de forma anómala en el plano cercano.

Pendiente (Slope): Evalúa la coherencia de la perspectiva geométrica calculando la inclinación media absoluta de los carriles detectados. Dado que los carriles reales proyectados en una cámara tienden a fugar hacia el horizonte con ángulos específicos, monitorear la pendiente media permite confirmar que el algoritmo está rastreando las líneas longitudinales de la vía y no falsos positivos transversales.

Método 1

Algoritmo Original

- Consistencia y detección de líneas baja en el pipeline Diurno.
- Valores aceptables en el pipeline Nocturno.

	Metric	Day Time (SHT)	Night Time (PHT)
0	Consistency	0.27	0.51
1	Distance (px)	1065.44	1009.56
2	Slope	1.17	0.77
3	Detection of Lines	0.36	0.65

Método 2

Algoritmo Modificado con PHT

-Se ha implementado el pipeline diurno con Hough Probabilístico

-Mejora de Consistencia y
Detección de líneas

	Metric	Day Time (PHT)	Night Time (PHT)
0	Consistency	0.59	0.51
1	Distance (px)	1064.01	1009.56
2	Slope	1.09	0.77
3	Detection of Lines	0.72	0.65

Método 3

- Desarrollo de Tracker

- Al observar estos resultados, nos preguntamos si es posible la mejora de los datos obtenidos y hemos desarrollado un modelo de post-procesamiento basado en Memoria y seguimiento temporal.
-

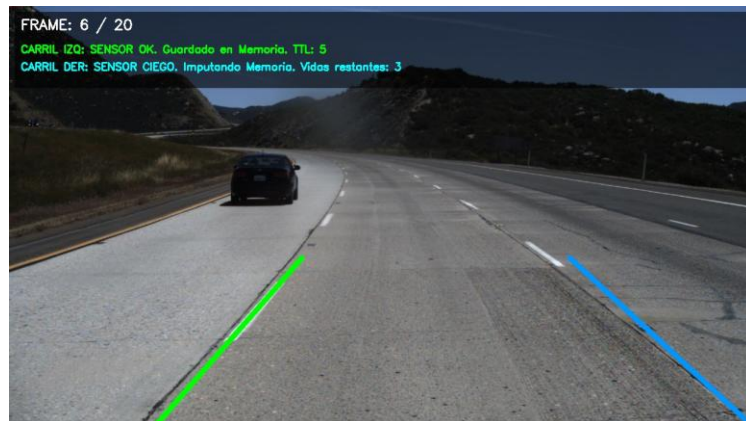
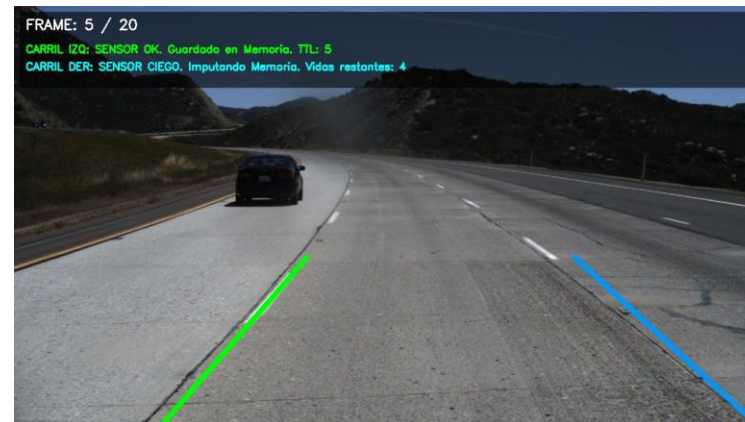
Tracker TTL

Tracker TTL: Imputación con Prevención de Deriva

Módulo de memoria a corto plazo para mitigar la ceguera temporal de la cámara sin comprometer la seguridad.

- **Sensor Activo (Visión Pura):** El sistema lee la carretera, actualiza la trayectoria real y recarga el contador a **5 vidas**.
- **Ceguera Temporal (Flickering):** Ante destellos o sombras, el sensor falla. El Tracker asume el control e **imputa la última trayectoria conocida** (resta 1 vida).
- **Prevención de Deriva (Seguridad):** Si la ceguera supera los 5 fotogramas (~160 ms), **la memoria se purga**. Se evita conducir a ciegas en curvas pronunciadas.

```
def postprocesar_tracker(m_izq_raw, c_izq_raw,
m_der_raw, c_der_raw, tracker):
    if m_izq_raw is not None:
        tracker['m_izq'], tracker['c_izq'] =
m_izq_raw, c_izq_raw
        tracker['ttl_izq'] = MAX_VIDAS
        m_izq_final, c_izq_final = m_izq_raw,
c_izq_raw
    else:
        if tracker['ttl_izq'] > 0:
            m_izq_final, c_izq_final =
tracker['m_izq'], tracker['c_izq']
            tracker['ttl_izq'] -= 1
        else:
            m_izq_final, c_izq_final = None,
None
```



FRAME: 5 / 20

CARRIL IZQ: SENSOR OK. Guardado en Memoria. TTL: 5
CARRIL DER: SENSOR OK. Guardado en Memoria. TTL: 5



FRAME: 6 / 20

CARRIL IZQ: SENSOR OK. Guardado en Memoria. TTL: 5
CARRIL DER: SENSOR CIEGO. Imputando Memoria. Vidas restantes: 4



FRAME: 7 / 20

CARRIL IZQ: SENSOR OK. Guardado en Memoria. TTL: 5
CARRIL DER: SENSOR CIEGO. Imputando Memoria. Vidas restantes: 3



FRAME: 8 / 20

CARRIL IZQ: SENSOR OK. Guardado en Memoria. TTL: 5
CARRIL DER: SENSOR CIEGO. Imputando Memoria. Vidas restantes: 2



FRAME: 9 / 20

CARRIL IZQ: SENSOR OK. Guardado en Memoria. TTL: 5
CARRIL DER: SENSOR CIEGO. Imputando Memoria. Vidas restantes: 1



FRAME: 10 / 20

CARRIL IZQ: SENSOR OK. Guardado en Memoria. TTL: 5
CARRIL DER: SENSOR CIEGO. Imputando Memoria. Vidas restantes: 0



FRAME: 11 / 20

CARRIL IZQ: SENSOR OK. Guardado en Memoria. TTL: 5
CARRIL DER: ALERTA: Sensor ciego. Memoria Agotada. Se??al Perdida.



FRAME: 12 / 20

CARRIL IZQ: SENSOR OK. Guardado en Memoria. TTL: 5
CARRIL DER: ALERTA: Sensor ciego. Memoria Agotada. Se??al Perdida.



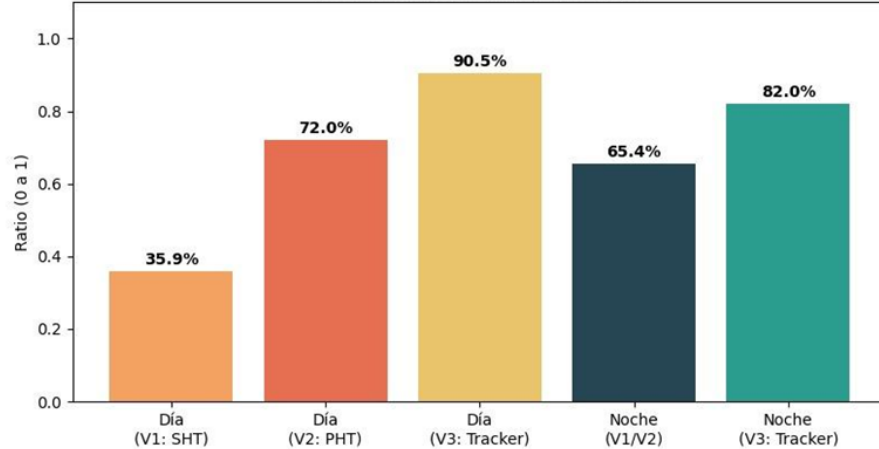
Resultados finales y comparativa

Observamos una mejora sustancial en la detección de líneas así como en la consistencia proporcionandonos unos datos muy buenos en comparación a las versiones anteriores.

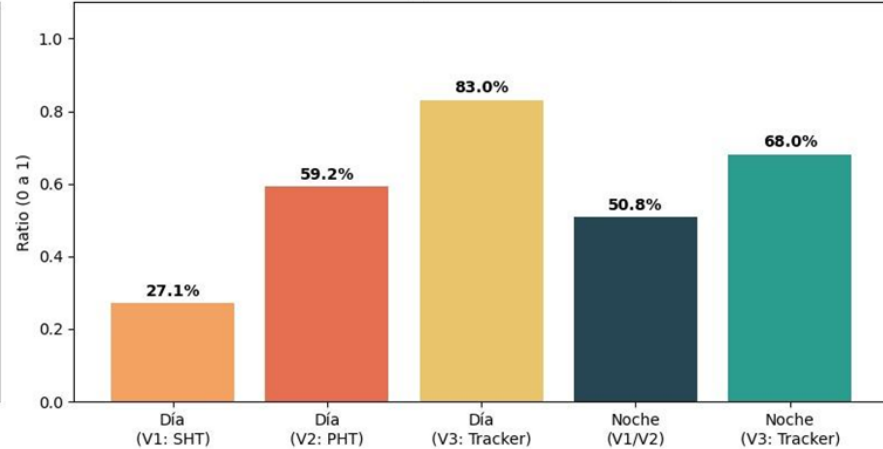
	Metric	Día (V1: SHT)	Día (V2: PHT)	Día (V3: PHT + Tracker)	Noche (V1/V2: PHT)	Noche (V3: PHT + Tracker)
0	Consistency	0.27	0.59	0.83	0.51	0.68
1	Distance (px)	1065.44	1064.01	1055.32	1009.56	1010.50
2	Slope (Mean)	1.17	1.09	1.15	0.77	0.77
3	Detection Rate	0.4	0.7	0.9	0.7	0.8

Evolución del Rendimiento por Arquitectura (Pipeline)

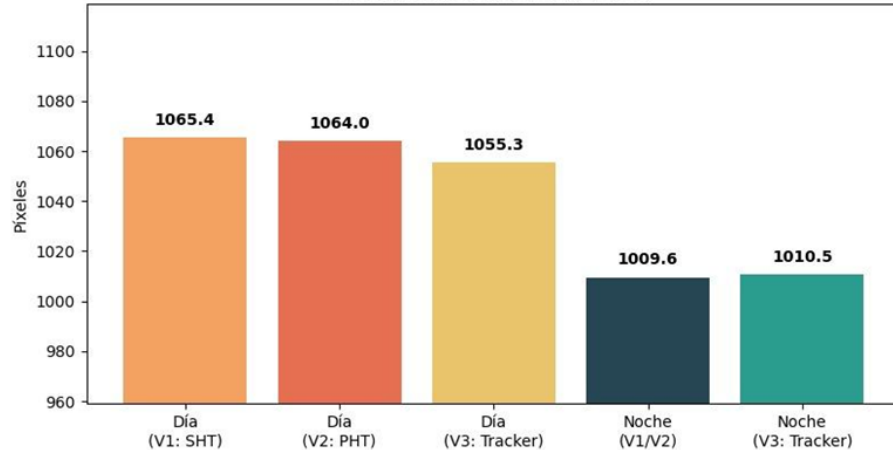
Detection Rate (Tasa de Éxito)



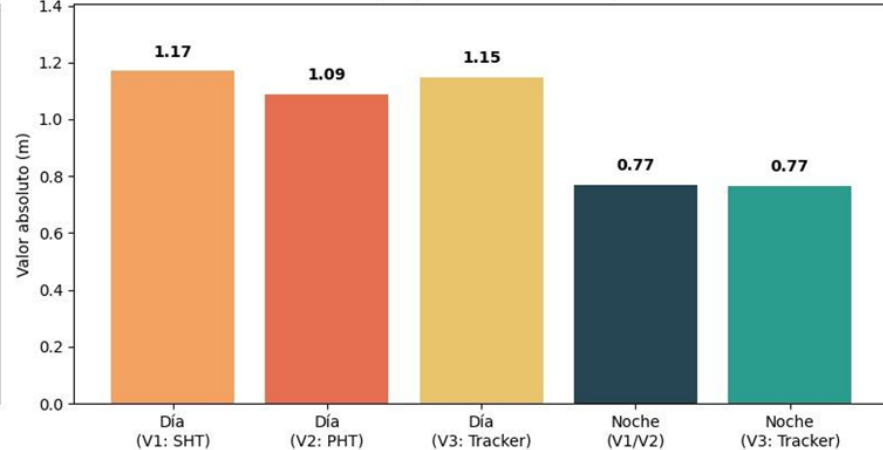
Consistency (Estabilidad sin deriva)



Distance (Distancia en base)



Slope (Inclinación Media)



CONCLUSIÓN

Muchas Gracias